Lean proofs and finite witnesses for the two papers on Steiner bicolorings

Juan Pablo Traverso Gianini
Independent researcher, Santiago, Chile
jtraverso@gmail.com
ORCID: 0009-0003-6068-4096

Technical companion to Papers I and II.
Preprint 1.0 — 6 November 2026.

This companion identifies the proof sources used for the two manuscripts. It also records how to distinguish their verification claims. The spectrum theorem is a theorem for all sufficiently large admissible profiles. The class-size characterization is a theorem for every positive profile; its proof includes an explicit finite table calculation.

The principal result of Paper I is cdr_type_spectrum. The auxiliary cdr_eventually calls T on the complete host (Proposition 3.2). Paper II proves existence at every order. The written proof invokes CDR 1999 for its last finite cases; the archived formal proof instead checks explicit multiplication tables.

1. Sources, entry point and build scope

The mathematical baseline is CDR revision 7643193 [S1]. The digest identifying this Git commit is **76431935a1d61dbd7d512f32c941e37de38f6ba0**. Its recorded build completed 10,331 Lake jobs. Use this revision to reproduce the recorded 6/7 criterion and finite base. Build record [S2] · Axiom audit [S3].

The baseline uses Lean 4.35.0-rc3 and Mathlib c55e6e786f49471c72fbddbec5415808896aec1e. The Lake configuration lists dependencies at local paths. In particular, CDR uses the 999-module import closure selected from Paper III. The provenance record distinguishes original source hashes from hashes after the compiler resource settings were inserted. The comparison with the released Paper III archive detects changes to historical documentation comments as well as inserted resource options. No mathematical code changes were found in the 999 checked modules. A separate 27-module closure for counting uses renamed import paths. These are dependencies of the CDR build. Paper III has its own published artifact. Dependency record · Paper III (DOI 10.5281/zenodo.23867836).

2. Reading the formal proof of the spectrum

The prefix for the following short names is CDRBicolour.Absorption, with the subnamespace indicated by its linked source. The library root imports the results below. The report measures the axioms of the declarations themselves.

Mathematical step	Formal source
Theorem 1, both directions; exact arithmetic predicate	**Corners.Spectrum** (`CDRBicolour/Absorption/Corners/Spectrum.lean`), declaration **cdr_type_spectrum**
Vertex balance, quotient and remainder, degree-zero obstruction	**Corners.Necessary** (`CDRBicolour/Absorption/Corners/Necessary.lean`)
Two alternative decompositions and the exact interior count	**Trade** (`CDRBicolour/Absorption/Exact/Trade.lean`), **Exact.Final** (`CDRBicolour/Absorption/Exact/Final.lean`), **cdr_aac_exact**
Planting, repairing the seed and completing with three types	**Corners.LowerT** (`CDRBicolour/Absorption/Corners/LowerT.lean`), **corner_lower**; **Seed3** (`CDRBicolour/Absorption/Corners/Seed3.lean`)
Extending one packing or prescribing the intersection of two systems	**Completion** (`CDRBicolour/Absorption/Completion.lean`), **cdr_completion**; **Refined.Completion** (`CDRBicolour/Absorption/Refined/Completion.lean`), **cdr_prescribed_intersection**
Separating Pasch counts and counting labeled systems	**PaschDiversity** (`CDRBicolour/Absorption/PaschDiversity.lean`), **cdr_many_pasch_counts**; **Wilson.Final** (`CDRBicolour/Absorption/Count/Wilson/Final.lean`), **realCount_log_asymptotic**

The standard axioms in the baseline report are propositional extensionality, classical choice and soundness of quotients: in Lean, propext, Classical.choice and Quot.sound. These are the only reported dependencies for the spectrum and the other asymptotic endpoints listed above. The mathematical article gives the point count, the unit trade, the use of T and T+, and the assembly of necessity and sufficiency. The longer inverse, planting, lattice and counting lemmas are given in the linked proof sources.

3. Reading the formal proof for every order

The endpoint is **cdr_characterization**, in namespace CDRBicolour. Its statement assumes the three sizes positive and gives equivalence between realization and admissibility. The route to sufficiency is independent of the use of Paper III in the spectrum article. **Characterization** (`CDRBicolour/Characterization.lean`). The necessity direction, admissible_of_realises, uses the conditions of Dinitz–Stinson, Proposition 1.2. The source paths below refer to the shared ZIP.

Mathematical step	Formal source
Product, class roles and composite descent	**Product** (`CDRBicolour/Product.lean`); **Descent.L165** (`CDRBicolour/Descent/L165.lean`)
Equitable edge coloring and final perfect matchings	**Master.Equitable** (`CDRBicolour/Master/Equitable.lean`), **eqMaster_realises**
Gap four; graph left by the gap-two construction	**Master.L168** (`CDRBicolour/Master/L168.lean`); **L183.P2** (`CDRBicolour/L183/P2.lean`)
Pairing colors missing the same vertex; completion of the narrow family	**L184.Assembly** (`CDRBicolour/L184/Assembly.lean`); **P2RouteClosed** (`CDRBicolour/P2RouteClosed.lean`)
Exhaustive grid, individual witnesses and induction	**Base.Cover** (`CDRBicolour/Base/Cover.lean`); **data** (`CDRBicolour/Base/Data`); **Universal** (`CDRBicolour/Universal.lean`)

The grid coverage uses a kernel decision procedure. The three all-order endpoints in the baseline report use the three standard axioms **plus 49 native decision certificates**, one for each of the 49 witness tables. In this release we decoded all tables independently, checked every third point, its two inverse lookups and its color pattern, and compared the table list with the uncovered grid profiles. The check passed for **1,203,132 unordered pairs**. We also checked that the two printed halves of the trade agree with their formal source and cover the same 27 edges. The Python check is independent corroboration of the data. The native certificates remain dependencies of the Lean declarations.

4. One freeze, two axiom reports

The two papers use one source revision of CDR. Their endpoints have different dependency closures. The build recorded at that revision completed 10,331 Lake jobs. The archived queries of the compiled objects on the author's machine match the recorded axiom sets for all twelve endpoints. The audit for this edition rechecks those records and the source archive; it does not perform a fresh Lean compilation. A separate query found no axioms for the coverage declaration gridOK_eq. The two imported 6/7 declarations used only the three standard axioms. The independent finite calculation checked 49 tables, of maximum order 487. The largest profile is (154,156,177). The table in Section 5 of Paper II gives mutually exclusive counts for the 1,551 profiles. The profile (696,702,745), of order 2143, is realized by the equitable branch and needs no table.

Plantholt is registered as PALOMAR-2026-11-04-000016 v2 (5 November), checked by Comparator, nanoda and con-ron. The dependency is the public main revision of the Plantholt repository, **f8900944da7743b1facd771139c9ebdbfd8aaa28**. It contains the Chetwynd–Hilton 1985 result and its use in Plantholt's criterion. For regular graphs on $2n$ points its hypothesis $12n \leq 7d$ is exactly $d \geq 6(2n)/7$. Dependency sources [S4].

To reproduce the audit, restore the pinned CDR revision and its path dependencies, including the Vizing shim and renamed counting closure, build the default Lake target, and run checks/AxiomsFreeze.lean and checks/SharedInputs.lean through Lake's Lean environment. The source package contains the outputs and a Python reader that verifies the finite data. These queries reused the completed local build. The archive includes the dependency sources with relative paths. This release records the archive and the completed build; a rebuild on a second machine is a separate experiment.

5. From the formal statements to the two proofs

Existence at large order. In Asymptotic.lean, cdr_eventually chooses the threshold before the three class sizes. The complete graph on the partition supplies the host. The proof takes the maximum of 400 and the threshold returned by main_theorem_T with parameters $(1/32, 4, 6)$. It passes the four proofs cdr_H1, cdr_H2, cdr_H3' and cdr_H4 to T. The last conversion, realises_of_decomposition, turns the triangle decomposition into the required Steiner system. This proves the auxiliary existence statement of Proposition 3.2.

The four host checks can be read separately. **H1** (`CDRBicolour/Absorption/Host.lean`) · **H2** (`CDRBicolour/Absorption/Lattice.lean`) · **H3/H4** (`CDRBicolour/Absorption/Hinge.lean`) · **Proposition 3.2** (`CDRBicolour/Absorption/Asymptotic.lean`).

Exact counts. Corners.Necessary starts from an actual system and counts the pairs through one vertex. Corners.Spectrum packages the resulting two tests in ExactNec. Its sufficiency proof first tests the planting inequality in the original order. If it holds, Graphs constructs the internal degrees, Plant assigns the third vertices and Lower assembles the minority packing. Residual computes the remaining neighbor counts; Seed3 and Lattice3 establish the fractional

and integer conditions on those edges. LowerT calls T with only the majority types. If the test holds after swapping the classes, Symmetry transports the construction back. If both tests fail, large_counts_R puts all six counts in the range of Exact.Final, which calls T+ and switches the reserved trades. The criterion and its converse refer to the same integer t.

The proof of the exact spectrum uses the following intermediate certificates. **Degree sequence** (`CDRBicolour/Absorption/Corners/Graphs.lean`) · **Planting** (`CDRBicolour/Absorption/Corners/Plant.lean`) · **Integer lattice** (`CDRBicolour/Absorption/Corners/Lattice3.lean`) · **Mass correction** (`CDRBicolour/Absorption/Exact/Final.lean`) · **Case split** (`CDRBicolour/Absorption/Corners/Spectrum.lean`).

All orders. Characterization exposes the implication in each direction. For sufficiency, Universal sorts the classes and uses strong induction on their sum. Descent.L165 combines smaller systems at composite orders. Master.Equitable supplies five block types by successive edge colorings. For the gap-two family the packing leaves a graph on the last class; L183.P2 proves it can be colored with the required number of colors and L184.Assembly attaches the missing pairs. Base.Cover identifies the inputs that remain below 5059. A successful check of an example establishes a single realization. The separate coverage and induction theorems explain how those individual statements enter a universal conclusion.

The project ledger records the successive constructions. For the proofs described here the controlling sources are the frozen theorem statements and imports. The development notes on the endpoints are best read alongside Corners.Spectrum, which assembles both ends and the interior. GATES · Corner constructions.

6. Reproducing one witness and reading the audit

The witness files encode two base-64 digits for each unordered pair in lexicographic order. Decode the third point, check that it differs from both endpoints, and look up each of the other two pairs. Both lookups must return the missing endpoint. Finally check that the three labels take exactly two values. For the smallest table the output is $012, 034, 056, 136, 145, 235, 246$, precisely the seven blocks of Figure 1 in Paper I. Running the same checks over the 49 files tests 1,203,132 pairs. They can be run independently of Lean.

The first command checks the package, the recorded Lean audit and the finite witnesses from the publication repository. To repeat the declaration queries, extract the shared archive, restore the compiled project environment, and run the last two commands from its root. These queries read existing objects. A source-only installation must first acquire the pinned dependencies and compile them.

```
python scripts/verify_repository.py
```

```
lake env lean checks/AxiomsFreeze.lean
```

```
lake env lean checks/SharedInputs.lean
```

7. Statement surface and trust boundary

The definitions and statement headers below are transcribed from the frozen sources. Names are relative to their namespaces. The proofs are in the shared source archive.

```
def Realises (a b c : ℕ) : Prop :=
  ∃ (S : STS (Fin (a + b + c))) (col : Fin (a + b + c) → Fin 3),
    IsStrictBicolouring S col ∧
      classSize col 0 = a ∧ classSize col 1 = b ∧ classSize col 2 = c
```

```
def AdmissibleProfile (a b c : ℕ) : Prop :=
  0 < a ∧ 0 < b ∧ 0 < c ∧
  ((a + b + c) % 6 = 1 ∨ (a + b + c) % 6 = 3) ∧
  ((a : ℤ) ^ 2 + b ^ 2 + c ^ 2 - a * b - a * c - b * c = a + b + c) ∧
  ¬ (a = 2 ∧ b = 2) ∧ ¬ (a = 2 ∧ c = 2) ∧ ¬ (b = 2 ∧ c = 2)

def CDRConjecture : Prop :=
  ∀ a b c : ℕ, AdmissibleProfile a b c → Realises a b c

theorem cdr_conjecture : CDRConjecture

theorem cdr_characterization {a b c : ℕ} (ha : 0 < a) (hb : 0 < b) (hc : 0 < c) :
    Realises a b c ↔ AdmissibleProfile a b c

theorem cdr_characterization_bicolourable3 {a b c : ℕ} (ha : 0 < a) (hb : 0 < b) (hc :
0 < c) :
    CDR.Bicolourable3 (a + b + c) a b c ↔ AdmissibleProfile a b c

def LowerNec (a b c t : ℕ) : Prop :=
  ∃ d q r : Fin 3 → ℕ, (∀ X, r X < sz a b c X) ∧
    (∀ X, (d X : ℤ) = sz a b c X - 1 - sz a b c (X + 1) + 2 * q X) ∧
    (∀ X, sz a b c X * d X + 2 * r X = 2 * (sz a b c (X + 2) * q (X + 2) + r (X + 2)))
∧
    (∀ X, ¬ (d X = 0 ∧ (r X = 1 ∨ r X = 2))) ∧ c * q 2 + r 2 = t

def ExactNec (a b c t : ℕ) : Prop :=
  2 * t ≤ a * (a - 1) ∧ LowerNec a b c t ∧ LowerNec a c b (a * (a - 1) / 2 - t)

def AACSpec (a b c t : ℕ) : Prop :=
  ∃ S : STS (Fin (a + b + c)), IsStrictBicolouring S (col3 a b c) ∧ typeCount S (col3
a b c) 0 2 = t

theorem cdr_type_spectrum : ∃ n₀ : ℕ, ∀ a b c : ℕ, AdmissibleProfile a b c → n₀ ≤ a
+ b + c →
    ∀ t : ℕ, AACSpec a b c t ↔ ExactNec a b c t

theorem cdr_eventually :
```

∃ n_0 : ℕ, ∀ a b c : ℕ, AdmissibleProfile a b c → n_0 ≤ a + b + c → Realises a b c

Each native table check trusts the compiler and the executed reduction. The actual Lean version prints a distinct generated native axiom for each table. The independent Python reader does not remove these axioms. The Vizing shim satisfies the package name required by downstream modules; the actual Vizing declarations are provided by the vendored LeanPool sources.

Acknowledgments

The author is deeply grateful to his wife María Paz and his children Lucas, Juan Cristóbal, Francisca, Raimundo and Benjamín for their love, patience and support.

Evidence for Theorem 6.2

The prescribed-intersection theorem has a symbolic Lean proof for all sufficiently large admissible orders; it is not certified by a finite enumeration. Its two residual completions use disjoint residue classes. The elementary intersection criterion and the formal dependency guide are printed in Paper I, Appendix B. The unchanged source archive includes `checks/AxiomsIntersection.lean`. Run `lake build CDRBicolour.Absorption.Refined.Completion` and `lake env lean checks/AxiomsIntersection.lean` after restoring the pinned dependencies. The repository script `scripts/verify_intersection_evidence.py` checks source and archived-record consistency only; it does not execute Lean. The publication checks did not rerun the Lean build.

Declaration-level audit

Declaration (within CDRBicolour)	Reported axioms
`Absorption.Corners.cdr_type_spectrum`	3
`Absorption.cdr_eventually`	3
`Absorption.cdr_completion`	3
`Absorption.Refined.cdr_prescribed_intersection`	3
`Absorption.PaschSeed.cdr_many_pasch_counts`	3
`Absorption.PaschSeed.cdr_many_nonisomorphic`	3
`Absorption.cdr_aac_dense`	3
`Absorption.Exact.cdr_aac_exact`	3
`Absorption.Count.Wilson.realCount_log_asymptotic`	3
`cdr_conjecture`	3 + 49
`cdr_characterization`	3 + 49
`cdr_characterization_bicolourable3`	3 + 49

References and computational tools

C. J. Colbourn, J. H. Dinitz and A. Rosa, *Bicoloring Steiner Triple Systems* (1999), Theorem 2.6. https://doi.org/10.37236/1457. J. P. Traverso Gianini, *Triangle decompositions III* (2026), v1.0. https://doi.org/10.5281/zenodo.23867836.

Claude, by Anthropic, and ChatGPT/Codex, by OpenAI, were used to explore and check arguments and to prepare the manuscript.

Certo and Jacobian were used to explore finite instances, check identities and assess candidate constructions. These computations informed the development of the arguments; finite checks were not treated as proofs of universal statements.

The author remains responsible for the arguments, citations, code and presentation. No AI system is listed as an author.

Dependency register. CCJS: PALOMAR-2026-11-04-000009 v2. Dinitz–Stinson: PALOMAR-2026-11-04-000014. Rudašová–Soták: PALOMAR-2026-11-05-000001. Vizing: PALOMAR-2026-10-15-000004. Counting: PALOMAR-2026-10-22-000017 v2. BKLS: PALOMAR-2026-11-06-000002 v1, registered on 6 November 2026 [S5]. Its source revision is 736c78e, the same revision used in the frozen dependency.

Supplementary material

[S1] CDR revision 7643193. https://github.com/jtraverso/cdr-steiner-preprints/blob/v1.0/archives/CDR_shared_Lean_source_7643193.zip.

[S2] Build record. https://github.com/jtraverso/cdr-steiner-preprints/blob/v1.0/publication/audit/EXISTING_BUILD.json.

[S3] Axiom audit. https://github.com/jtraverso/cdr-steiner-preprints/blob/v1.0/publication/audit/live_axioms.txt.

[S4] Dependency sources. https://github.com/jtraverso/plantholt-overfull-criterion/tree/f8900944da7743b1facd771139c9ebdbfd8aaa28.

[S5] BKLS public registration. https://palomar-registry.org/entry?id=PALOMAR-2026-11-06-000002&version=1.